DEVELOPMENT OF CHATBOT USING PYTHON

Building a chatbot with object detection capabilities using the YOLO (You Only Look Once) machine learning algorithm involves several steps, including setting up the YOLO model, training it on a dataset, integrating it with a chatbot interface, and evaluating its performance

Prerequisites:

Python: Make sure you have Python installed on your system. You can download it from python.org.

YOLO Model: You need the YOLO pre-trained model weights and configuration files. You can find them on the official YOLO website or repository.

Deep Learning Libraries: Install required libraries using pip.

pip install opencv-python numpy

Required Steps:

1. Setting Up YOLO Model:

Download the YOLO pre-trained model files (yolov3.cfg, yolov3.weights, and coco.names) and place them in a directory.

2. Object Detection Code:

Create a Python script for object detection using YOLO. Here's a basic example using OpenCV and the YOLO model.

```
import cv2
import numpy as np
# Load YOLO
net = cv2.dnn.readNet("yolov3.weights", "yolov3.cfg")
```

```
classes = []
with open("coco.names", "r") as f:
classes = f.read().strip().split('\n')

layer_names = net.getLayerNames()
output_layers = [layer_names[i[0] - 1] for i in
net.getUnconnectedOutLayers()]

def detect_objects(img_path):
# Load image
img = cv2.imread(img_path)
img = cv2.resize(img, None, fx=0.4, fy=0.4) height, width, channels =
img.shape

# Detecting objects
 blob = cv2.dnn.blobFromImage(img, 0.00392, (416, 416), (0, 0, 0), True,
crop=False)
net.setInput(blob)
outs = net.forward(output_layers)

# Information to show on the screen
class_ids = []
confidences = []
boxes = []

# Interpret the output
for out in outs:
  for detection in out:
    scores = detection[5:]
    class_id = np.argmax(scores)
     confidence = scores[class_id]
  if confidence > 0.5:
      # Object detected
      center_x = int(detection[0] * width)
      center_y = int(detection[1] * height)
```

```
w = int(detection[2] * width)
h = int(detection[3] * height)

# Rectangle coordinates
x = int(center_x - w / 2)
y = int(center_y - h / 2)

boxes.append([x, y, w, h])
confidences.append(float(confidence))
class_ids.append(class_id)

indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)

detected_objects = []
for i in range(len(boxes)):
  if i in indexes:
    label = str(classes[class_ids[i]])
    confidence = confidences[i]
    detected_objects.append({"label": label, "confidence":
confidence})

  return detected_objects
```

3. Integrating with Chatbot:

Now, integrate the object detection code with your chatbot. Here's a basic example of a chatbot that utilizes the object detection function.

```
# Import required libraries for chatbot from flask import Flask,
request, jsonify
 import json

app = Flask(__name__)
# Endpoint for chatbot
```

```
@app.route('/chat', methods=['POST'])
    def chat():
        data = request.get_json()
        message = data['message']

# Perform object detection if the message is related to images
        if "image" in message:
          image_path = message["image"]
          detected_objects = detect_objects(image_path)
          response = {"objects": detected_objects}
else:
        response = {"message": "Chatbot received: " + message}
return jsonify(response)

if __name__ == '__main__':
app.run(port=5000)
```

4. Running the Chatbot:

Run your Flask application:
python app.py

If you send a message containing an image to the chatbot, it will detect objects in the image and return the labels and confidence scores of the detected objects.